explains how residuals can be used to precompute policy fragments.

These static predicates can be converted into efficient database or graph filters. For example:

- `principal.department == resource.department` becomes a join condition between the principal and resource tables.
- `resource.shareable == true` becomes an indexed field constraint in the search index.
- `principal in resource.viewers` becomes an edge traversal in the ReBAC graph.

When searching the graph, the UI uses this scope to prefilter candidates before calling the PDP, for example, returning only documents tagged as shareable or users from an approved email domain.

**3. Confirm with the PDP**

After filtering the graph results, C³ contacts the tenant PDP to confirm each candidate with a complete Cedar evaluation.

Batch evaluation allows the PDP to check all remaining resources or principals in a single call, applying any dynamic conditions that depend on runtime context, such as time windows, session risk, or (for ACME employees) device posture.

Only items that return `permit` are displayed in the UI.

Here’s an example of how these three steps could be implemented in Python with a hypothetical graph database SDK:

```
# Example: docs this user can access
cands   = graph.traverse_from_principal(p, action="doc:view")
scoped  = prefilter_by_scope(cands, action="doc:view")
result  = pdp.batch_authorize(principal=p, action="doc:view", reso
urces=scoped, context=ctx)
docs    = [r for r, dec in result.items() if dec == "Permit"]
```